

ASSIGNMENT 1.3

Name: P. Niharika

HT NO: 2303A51694

Batch: 24

Task 1: AI-Generated Logic Without Modularization (String Reversal Without Functions)

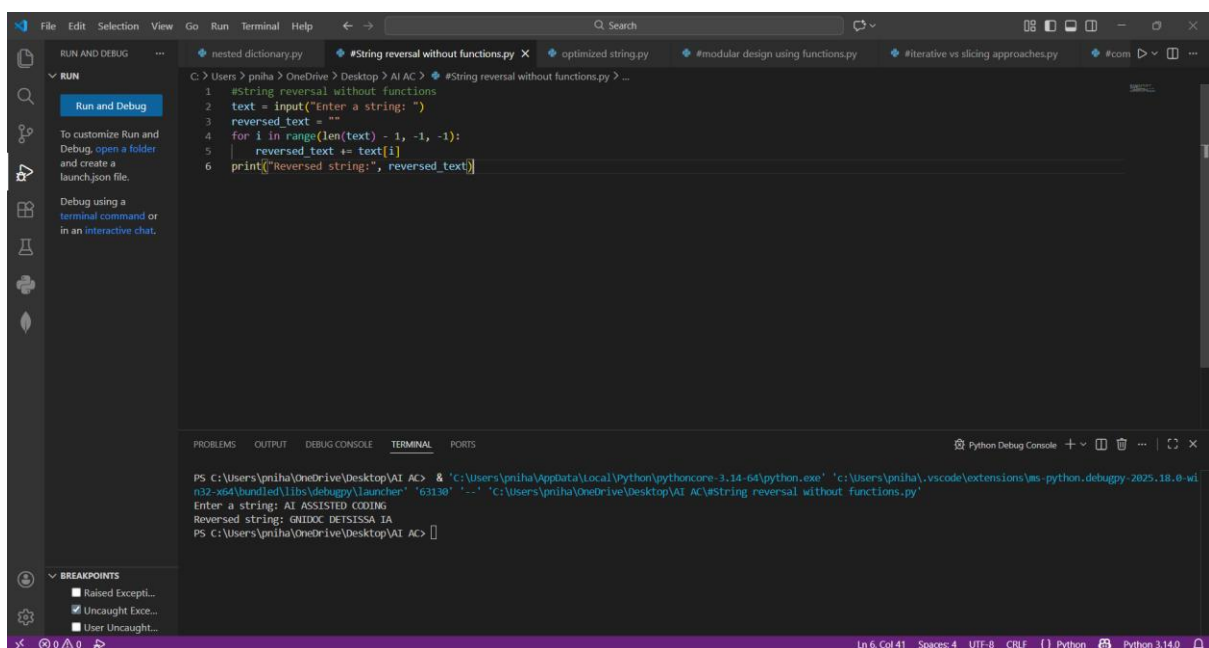
❖ Scenario

You are developing a basic text-processing utility for a messaging application.

❖ Task Description

Use GitHub Copilot to generate a Python program that:

- Reverses a given string
 - Accepts user input
 - Implements the logic directly in the main code
 - Does not use any user-defined functions
- #### ❖ Expected Output
- Correct reversed string
 - Screenshots showing Copilot-generated code suggestions
 - Sample inputs and outputs



The screenshot displays the Visual Studio Code interface. The editor window shows a Python file named `#String reversal without functions.py` with the following code:

```
1 #String reversal without functions
2 text = input("Enter a string: ")
3 reversed_text = ""
4 for i in range(len(text) - 1, -1, -1):
5     reversed_text += text[i]
6 print("Reversed string:", reversed_text)
```

The terminal window at the bottom shows the execution of the script. The command prompt is `PS C:\Users\pnih\OneDrive\Desktop\AI AC>`. The user enters the string `AI ASSISTED CODING`. The output is `Reversed string: GNIDOC DETSISSA IA`. The status bar at the bottom indicates the file is at line 6, column 41, using UTF-8 encoding with CRLF line endings, and the Python version is 3.14.0.

Task 2: Efficiency & Logic Optimization (Readability Improvement)

❖ Scenario

The code will be reviewed by other developers.

❖ Task Description

Examine the Copilot-generated code from Task 1 and improve it by:

- Removing unnecessary variables
- Simplifying loop or indexing logic
- Improving readability
- Use Copilot prompts like:
 - “Simplify this string reversal code”
 - “Improve readability and efficiency”

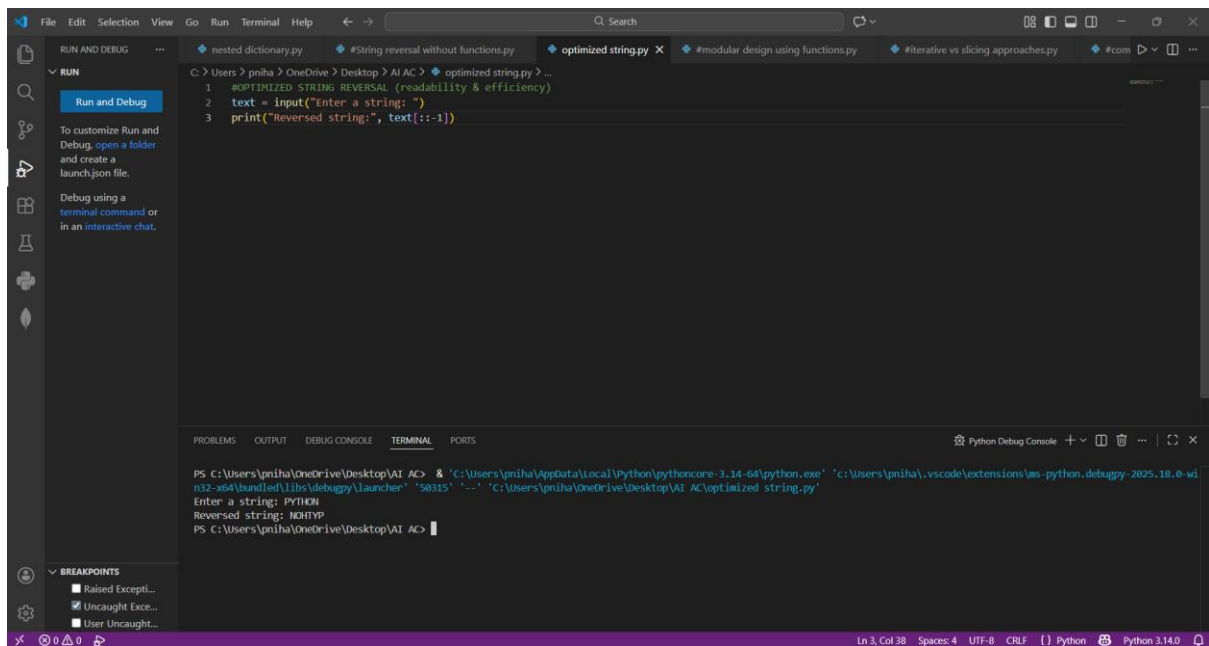
Hint:

Prompt Copilot with phrases like

“optimize this code”, “simplify logic”, or “make it more readable”

❖ Expected Output

- Original and optimized code versions
- Explanation of how the improvements reduce time complexity



The screenshot shows the Visual Studio Code interface. The editor has several tabs open: 'nested dictionary.py', '#String reversal without functions.py', 'optimized string.py', '#modular design using functions.py', '#iterative vs slicing approaches.py', and '#com'. The 'optimized string.py' tab is active, displaying the following code:

```
1 #OPTIMIZED STRING REVERSAL (readability & efficiency)
2 text = input("Enter a string: ")
3 print("Reversed string:", text[::-1])
```

The bottom panel shows the 'TERMINAL' tab with the following output:

```
PS C:\Users\pnha\OneDrive\Desktop\AI AC> & 'C:\Users\pnha\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'C:\Users\pnha\.vscode\extensions\ms-python.debugpy-2025.18.0-wi
n32-x64\bundled\libs\debugpy\launcher' '58315' '-' 'C:\Users\pnha\OneDrive\Desktop\AI AC\optimized_string.py'
Enter a string: PYTHON
Reversed string: NOHTYP
PS C:\Users\pnha\OneDrive\Desktop\AI AC> |
```

The status bar at the bottom indicates 'Ln 3, Col 38', 'Spaces: 4', 'UTF-8', 'CRLF', and 'Python 3.14.0'.

Task 3: Modular Design Using AI Assistance (String Reversal Using Functions)

❖ Scenario

The string reversal logic is needed in multiple parts of an application.

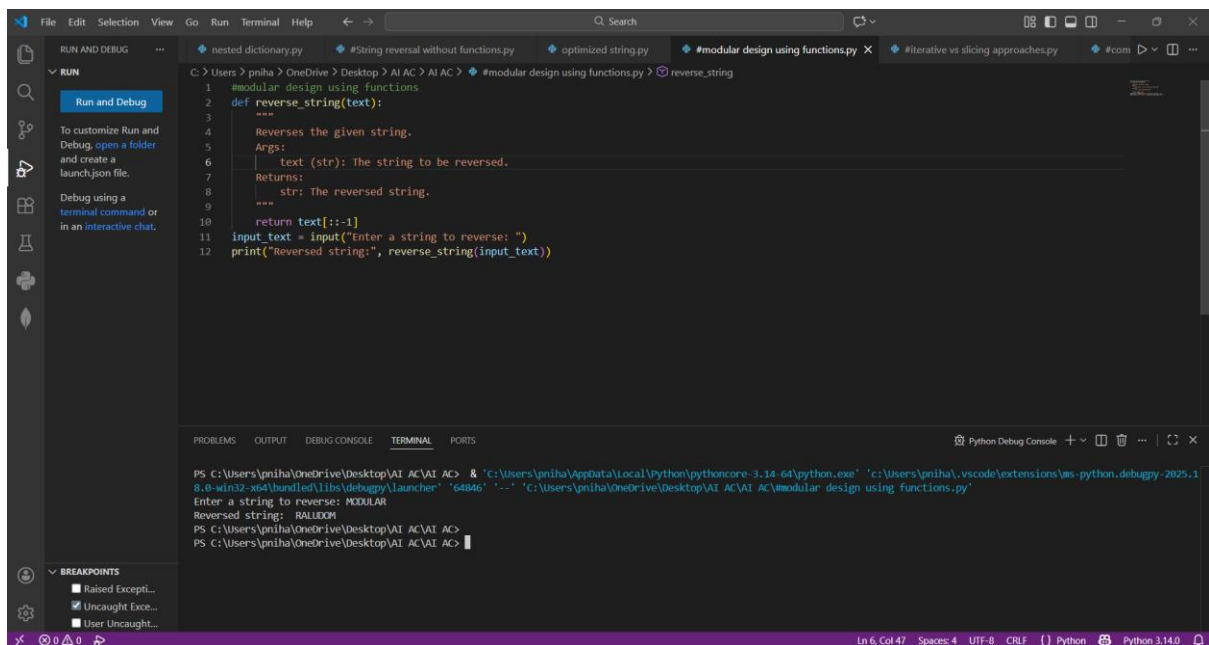
❖ Task Description

Use GitHub Copilot to generate a function-based Python program that:

- Uses a user-defined function to reverse a string
- Returns the reversed string
- Includes meaningful comments (AI-assisted)

❖ Expected Output

- Correct function-based implementation
- Screenshots documenting Copilot's function generation
- Sample test cases and outputs



The screenshot shows a Visual Studio Code editor window with a Python file named `reverse_string.py`. The code defines a function `reverse_string(text)` that reverses a string and prints the result. The terminal output shows the program being executed, with the input string `MODULAR` and the reversed string `RALUDOM`.

```
1 #modular design using functions
2 def reverse_string(text):
3     """
4     Reverses the given string.
5     Args:
6     | text (str): The string to be reversed.
7     Returns:
8     | str: The reversed string.
9     """
10    return text[::-1]
11    input_text = input("Enter a string to reverse: ")
12    print("Reversed string:", reverse_string(input_text))
```

Terminal Output:

```
PS C:\Users\pnha\OneDrive\Desktop\AI AC\AI AC> & "C:\Users\pnha\AppData\Local\Python\pythoncore-3.14-64\python.exe" "C:\Users\pnha\.vscode\extensions\ms-python.debugpy-2025.18.0-x64\bin\debugpy_launcher" "64846" "--" "C:\Users\pnha\OneDrive\Desktop\AI AC\AI AC\modular design using functions.py"
Enter a string to reverse: MODULAR
Reversed string: RALUDOM
PS C:\Users\pnha\OneDrive\Desktop\AI AC\AI AC>
```

Task 4: Comparative Analysis – Procedural vs Modular Approach (With vs Without Functions)

❖ Scenario

You are asked to justify design choices during a code review.

❖ Task Description

Compare the Copilot-generated programs:

- Without functions (Task 1)
- With functions (Task 3)

Analyze them based on:

- Code clarity
- Reusability
- Debugging ease
- Suitability for large-scale applications
- ❖ Expected Output

Comparison table or short analytical report

The screenshot shows a Python IDE with a file named `iterative vs slicing approaches.py`. The code implements two methods for reversing a string: an iterative loop-based method and a slicing-based method. The terminal output shows the program running successfully for two inputs: 'HELLO WORLD' and 'PYTHON'.

```

1 #iterative vs slicing approaches
2 #iterative (loop-based)
3 text = input("\nEnter a string to (iterative): ")
4 reversed_text = ""
5 for char in text:
6     reversed_text = char + reversed_text
7 print("Reversed string (iterative):", reversed_text)
8 #slicing (pythonic)
9 text = input("\nEnter a string to (slicing): ")
10 print("Reversed string (slicing):", text[::-1])
  
```

```

PS C:\Users\pnih\OneDrive\Desktop\AI AC\AI AC> & 'C:\Users\pnih\AppData\Local\python\pythoncore-3.14-64\python.exe' 'C:\Users\pnih\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '49677' '-' 'C:\Users\pnih\OneDrive\Desktop\AI AC\AI AC\iterative vs slicing approaches.py'

Enter a string to (iterative): HELLO WORLD
Reversed string (iterative): DLROW OLLEH

Enter a string to (slicing): PYTHON
Reversed string (slicing): NOHTYP
PS C:\Users\pnih\OneDrive\Desktop\AI AC\AI AC>
  
```

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches to String Reversal)

❖ Scenario

Your mentor wants to evaluate how AI handles alternative logic paths.

❖ Task Description

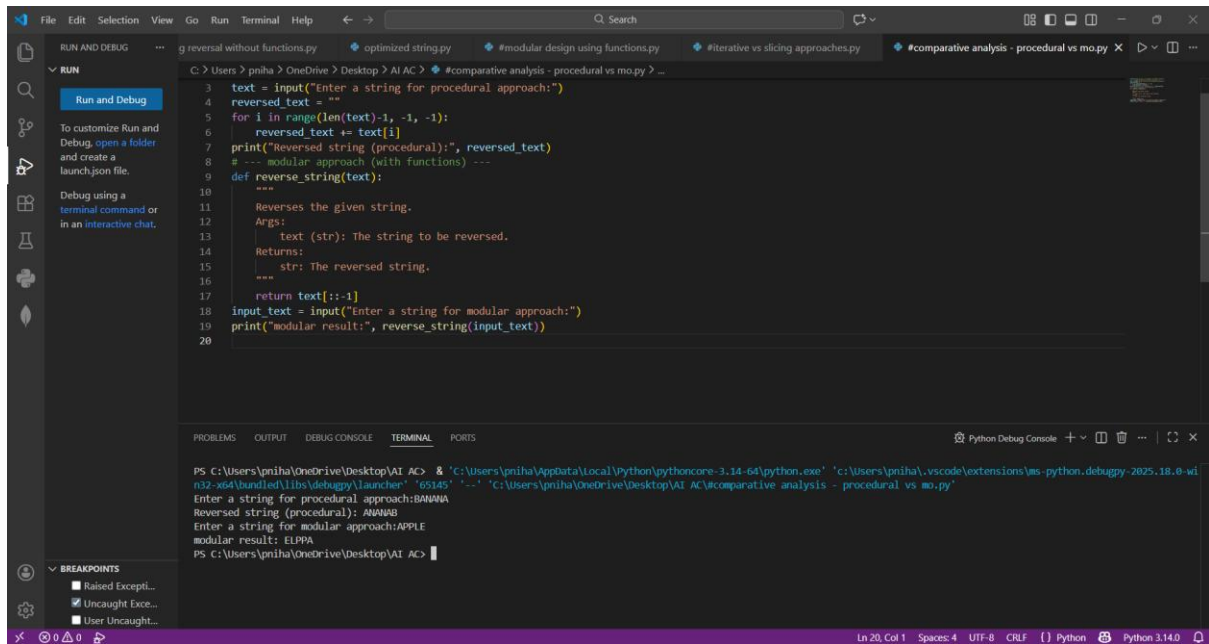
Prompt GitHub Copilot to generate:

- A loop-based string reversal approach
- A built-in / slicing-based string reversal approach

❖ Expected Output

- Two correct implementations
- Comparison discussing:
 - Execution flow
 - Time complexity

- Performance for large inputs
- When each approach is appropriate



The screenshot shows a Python IDE with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The code editor displays a Python script for reversing a string. The script includes a procedural approach using a loop and a modular approach using a function. The terminal shows the execution of the script, with input and output for both approaches.

```
3 text = input("Enter a string for procedural approach:")
4 reversed_text = ""
5 for i in range(len(text)-1, -1, -1):
6     reversed_text += text[i]
7 print("Reversed string (procedural):", reversed_text)
8 # --- modular approach (with functions) ---
9 def reverse_string(text):
10     """
11     Reverses the given string.
12     Args:
13         text (str): The string to be reversed.
14     Returns:
15         str: The reversed string.
16     """
17     return text[::-1]
18 input_text = input("Enter a string for modular approach:")
19 print("modular result:", reverse_string(input_text))
20
```

The terminal output shows the execution of the script:

```
PS C:\Users\pnih\OneDrive\Desktop\AI AC> & "c:\Users\pnih\AppData\Local\Python\pythoncore-3.14-64\python.exe" "c:\Users\pnih\.vscode\extensions\ms-python.debugpy-2025.18.0-wi
n32-x64\bundled\libs\debugpy\launcher" "65145" "--" "C:\Users\pnih\OneDrive\Desktop\AI AC\comparative analysis - procedural vs mo.py"
Enter a string for procedural approach: BANANA
Reversed string (procedural): ANANAB
Enter a string for modular approach: APPLE
modular result: ELPPA
PS C:\Users\pnih\OneDrive\Desktop\AI AC>
```